# PIN: Protocol Buffer-Based Input Normalization for Program Analysis and Testing

Priyatam Annambhotla

Dissertation submitted to the Faculty of the

Virginia Polytechnic Institute and State University

in partial fulfillment of the requirements for the degree of

Master of Science

in

Electrical and Computer Engineering

Binoy Ravindran, Chair

Zhoulai Fu, Co-chair

TBD

TBD

TBD

October 15, 2025

Blacksburg, Virginia

Keywords: Program Analysis, Input Normalization, Protocol Buffers, C Program Transformation, Fuzzing, Static Analysis

# PIN: Protocol Buffer-Based Input Normalization for Program Analysis and Testing

Priyatam Annambhotla

(ABSTRACT)

This thesis presents PIN (Program Input Normalization), a toolchain that transforms C programs so they accept standardized Protocol Buffer inputs. PIN automatically extracts input structures, synthesizes schemas, and emits nanopb-based wrappers that reconstruct the original calling convention. A new pointer management strategy classifies pointer parameters (scalars, slices, structs, chains) and generates helper messages and reconstruction logic for both the nanopb wrapper and a C++ reference runner. To reason about semantic equivalence when fuzzing arbitrary byte streams, we adopt an *equivalence modulo input validation* (EMI) policy: only inputs that decode into values satisfying the original function's preconditions are compared across implementations. Evaluation on coreutils, Toyota ITC, and custom pointer-heavy benchmarks shows that PIN covers 8/9 pointer fixtures end-to-end, improves test generation coverage, and exposes remaining gaps (struct slices, opaque pointers). The approach bridges manual input crafting and automated analysis by providing a scalable, semantics-aware normalization pipeline.

# PIN: Protocol Buffer-Based Input Normalization for Program Analysis and Testing

Priyatam Annambhotla

(GENERAL AUDIENCE ABSTRACT)

Software testing is crucial for ensuring programs work correctly, but creating the right test inputs for C programs is often difficult and time-consuming. This research introduces PIN, a tool that automatically converts C programs to accept standardized inputs using Google's Protocol Buffers. PIN not only learns the basic shapes of inputs, it also understands pointers—the links C programs use to reference data in memory—and regenerates them safely for testing. Because random byte streams can describe nonsense inputs, we compare behaviour only when the decoded data meets the program's expectations, an approach known as "equivalence modulo input validation." In effect, PIN builds a universal adapter that lets testing tools communicate with complex C software while respecting the program's rules. The tool successfully handles real-world programs and pointer-heavy examples, making systematic testing more practical for software companies, security researchers, and anyone who needs reliable C code.

# Dedication

*This is where you put your dedications.*

# Acknowledgments

This is where you put your acknowledgments.

# Contents

# List of Figures

# List of Tables

# Chapter 1

# Introduction

Software testing and program verification remain among the most critical challenges in software engineering. Despite decades of research and significant advances in automated testing techniques, the process of generating meaningful test inputs for complex programs continues to be a labor-intensive and error-prone task. This challenge is particularly acute for programs written in C, where manual memory management, pointer arithmetic, and complex data structures create barriers to automated analysis and testing.

Traditional approaches to program testing often require extensive manual effort to understand input formats, create test harnesses, and generate appropriate test data. This manual process not only limits the scalability of testing efforts but also introduces opportunities for human error and oversight. Moreover, the heterogeneous nature of input formats across different programs makes it difficult to apply systematic testing approaches uniformly.

## 1.1 Problem Statement

The primary challenge addressed in this thesis is the lack of a unified approach for automatically normalizing diverse input formats in C programs to enable systematic testing and analysis. Current testing tools and frameworks typically require manual specification of input structures or operate on specific file formats, limiting their applicability and scalability.

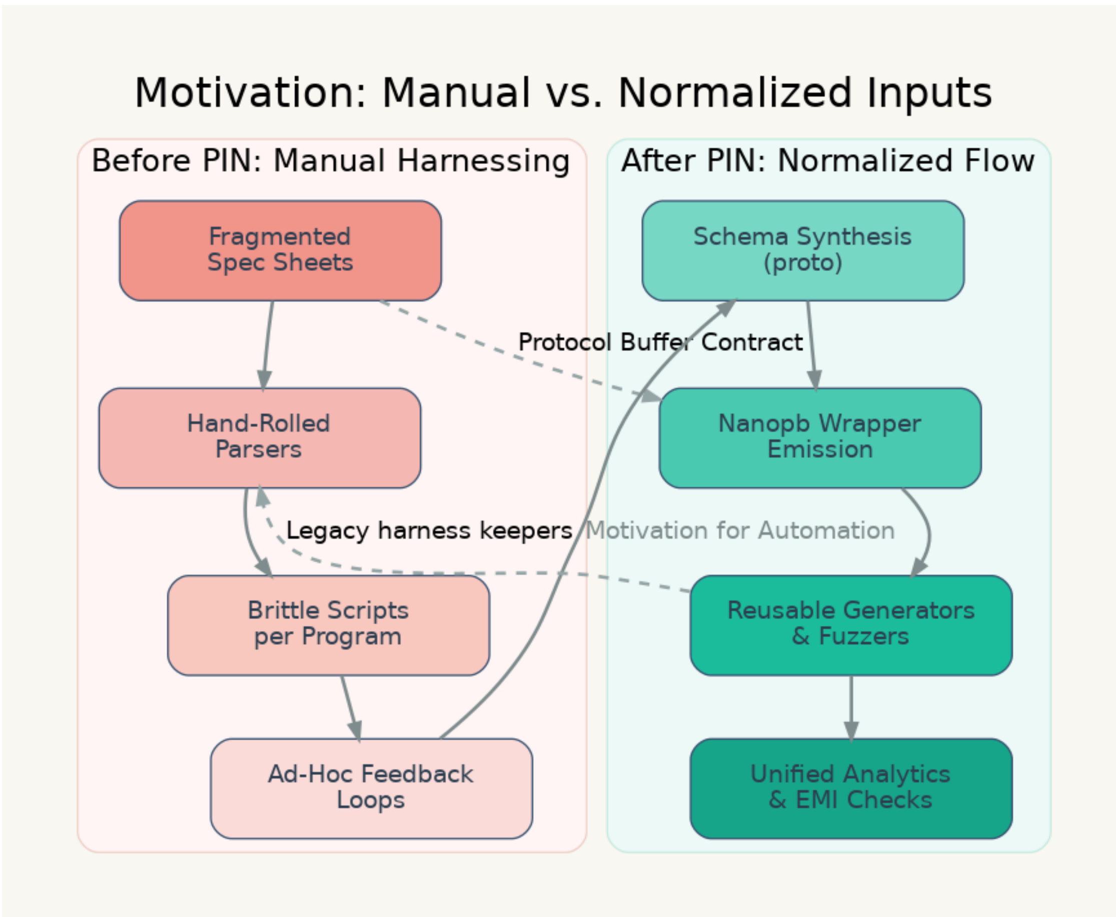


Figure 1.1: Contrasting the fragmented manual harnessing workflow with the normalized PIN-enabled flow underscores the thesis motivation.

Several specific problems contribute to this challenge:

**Input Format Diversity**: C programs accept inputs in countless formats, from simple command-line arguments to complex binary protocols. Each format requires specialized knowledge and custom tooling for effective testing.

**Manual Test Harness Creation**: Existing testing approaches often require manual creation of test harnesses and input generation logic, which is time-consuming and error-prone.

**Limited Automation**: The lack of standardized input representation prevents the development of generic testing tools that can work across different programs without modification.

**Analysis Integration Gaps**: Static analysis tools that extract program structure often cannot easily integrate with dynamic testing tools due to incompatible data representations.

## 1.2 Proposed Solution

This thesis presents PIN (Program Input Normalization), a novel tool that addresses these challenges by automatically transforming C programs to accept serialized inputs using Google Protocol Buffers. PIN bridges the gap between static program analysis and dynamic testing by providing a standardized input representation that enables systematic test generation and execution.

The key innovation of PIN lies in its ability to automatically analyze C source code, extract input structure information, and generate the necessary infrastructure for input normalization without requiring manual intervention. This approach enables the creation of generic testing workflows that can be applied uniformly across different C programs.

PIN's approach consists of several integrated components:

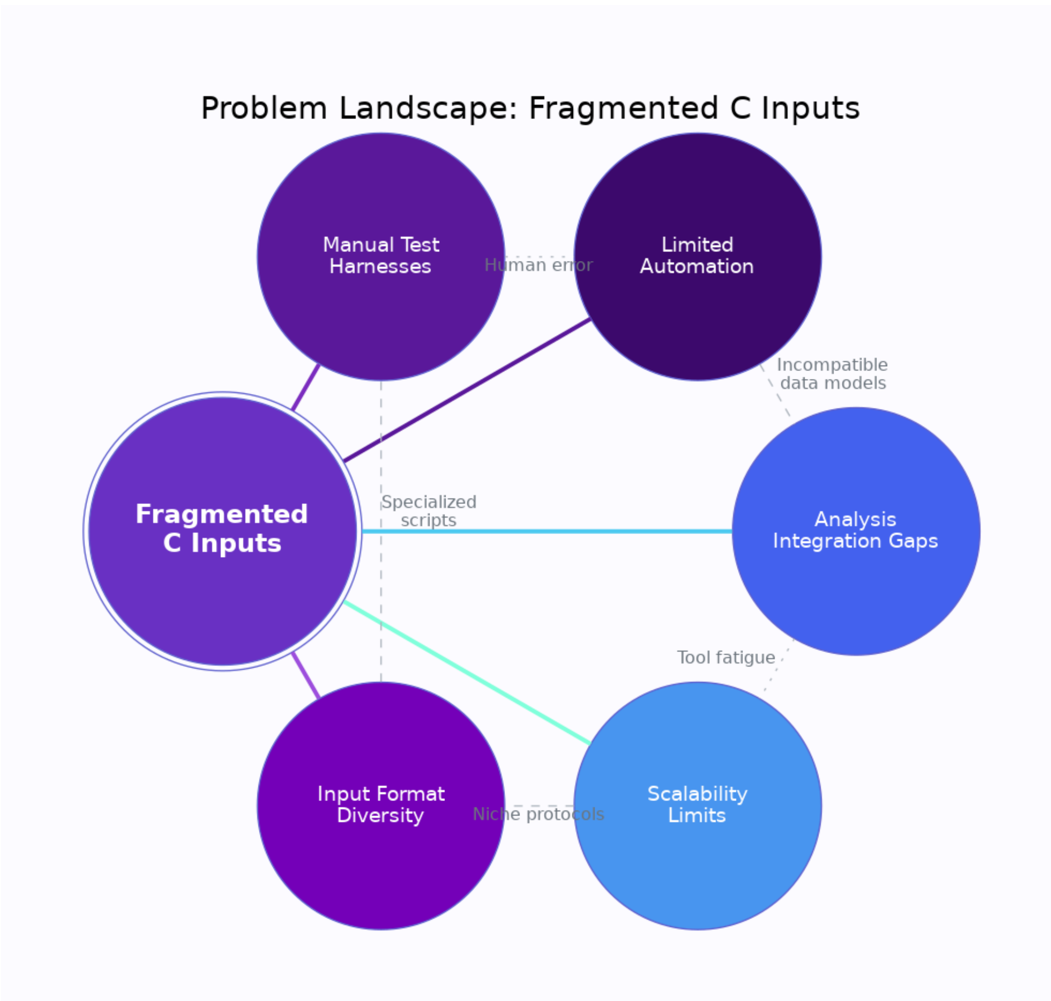


Figure 1.2: Centralizing the challenge around fragmented C inputs while highlighting surrounding pain points helps frame the problem statement.

1. **Automated Structure Extraction**: PIN uses static analysis techniques with both pycparser and libclang backends to automatically extract input structure information from C source code.

2. **Protocol Buffer Schema Generation**: The extracted structure information is automatically converted into Protocol Buffer schema definitions, providing a standardized serialization format.

3. **Wrapper Code Generation**: PIN generates wrapper code using nanopb that handles deserialization and calls the original program functions with properly formatted inputs.

4. **Testing Infrastructure**: The normalized representation enables the creation of generic test input generators and automated testing workflows.

## 1.3 Contributions

This thesis makes several significant contributions to the field of automated software testing and program analysis:

**Novel Input Normalization Approach**: PIN introduces the first automated approach for normalizing diverse C program inputs into a standardized Protocol Buffer representation, enabling uniform testing across different programs.

**Dual Parser Architecture**: The implementation supports both pycparser and libclang parsing backends, providing flexibility and robustness in handling different C language constructs and preprocessor configurations.

**Embedded Systems Compatibility**: By using nanopb for serialization, PIN maintains compatibility with resource-constrained environments typical in systems programming.

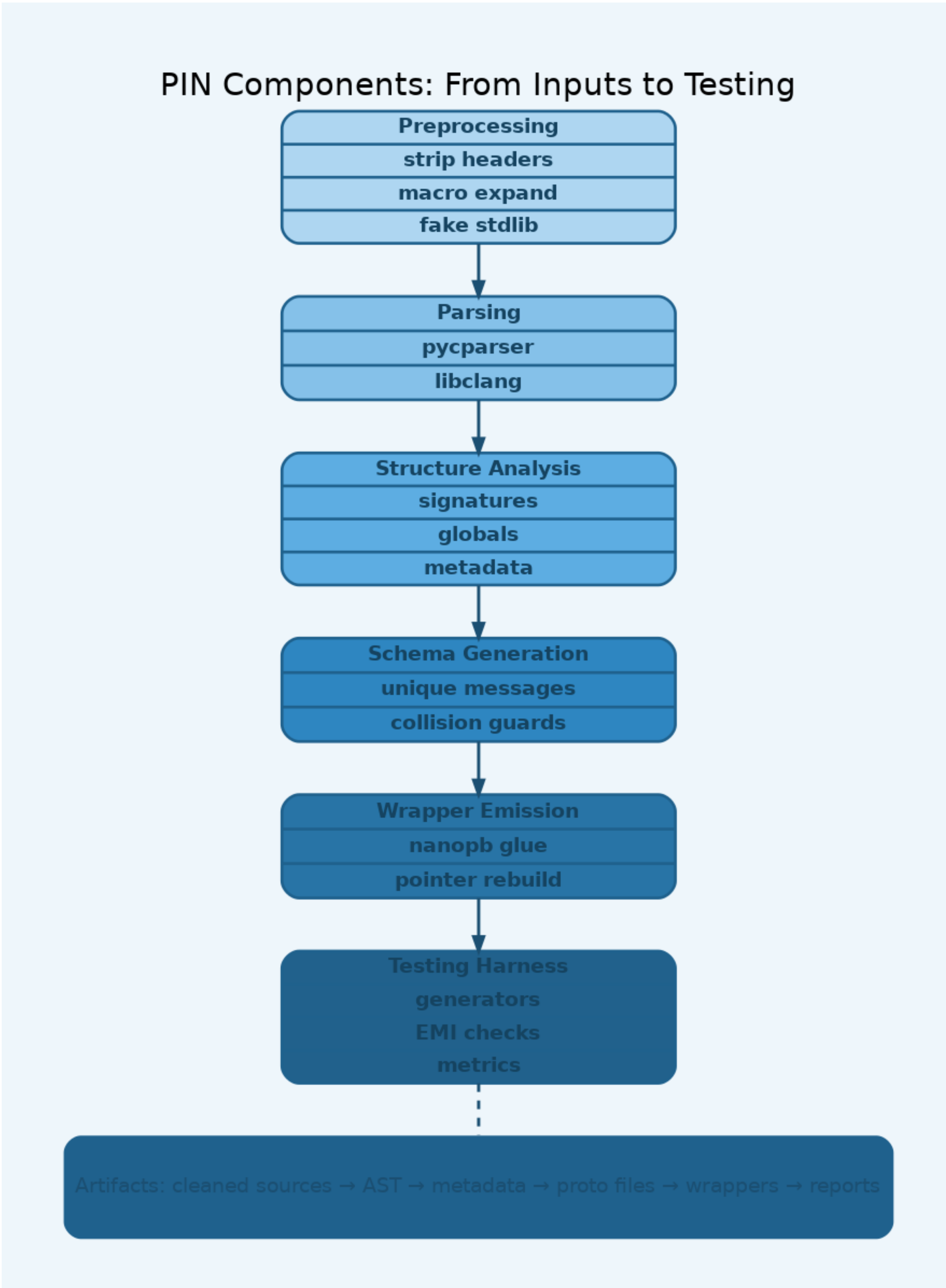


Figure 1.3: Stacked view of the four major PIN components and the artifacts they emit for subsequent stages.

**Pointer-Aware Reconstruction**: PIN contributes a reusable pointer metadata system and helper message library that reconstructs scalar, slice, struct, and pointer-chain arguments consistently across both the nanopb wrapper and a reference C++ runner.

**Equivalence Modulo Input Validation**: The evaluation introduces an EMI-based differential testing policy that compares behaviours only on decoded inputs satisfying the original function's preconditions, clarifying the semantics of "invalid" fuzzing cases.

**Comprehensive Evaluation**: The thesis presents a thorough evaluation of PIN using real-world C programs from coreutils, the Toyota ITC benchmark suite, and newly curated pointer fixtures, demonstrating practical applicability.

**Open Source Implementation**: PIN is provided as an open-source tool with comprehensive documentation, enabling adoption and further research by the community.

## 1.4 Thesis Organization

This thesis is organized as follows:

**Chapter 2** provides a comprehensive review of related work in program analysis, automated testing, fuzzing, and serialization approaches. This chapter establishes the theoretical foundation and identifies gaps that PIN addresses.

**Chapter 3** presents the design and implementation of PIN, including detailed descriptions of the parsing strategies, Protocol Buffer schema generation, wrapper code creation, and the overall pipeline architecture.

**Chapter 4** evaluates PIN's effectiveness through comprehensive experiments on real-world C programs, analyzing performance, coverage improvements, and limitations.

**Chapter 5** discusses the implications of the results, examines threats to validity, and compares PIN with existing approaches.

**Chapter 6** concludes the thesis by summarizing contributions and outlining directions for future work.

The appendices provide detailed implementation information, complete evaluation data, and examples of generated code to support reproducibility and further research.

# Chapter 2

# Review of Literature

This chapter provides a comprehensive review of the literature relevant to program input normalization, automated testing, and serialization approaches for C programs. The review is organized into several key areas that form the foundation for the PIN (Program Input Normalization) approach presented in this thesis.

## 2.1 Program Analysis and Testing Foundations

Static analysis and dynamic testing have long been fundamental approaches to software verification and validation. Aho et al. [1] established the theoretical foundations for program analysis through compiler construction techniques, particularly the use of abstract syntax trees (ASTs) for representing program structure. This work laid the groundwork for modern static analysis tools that can extract semantic information from source code.

The evolution of automated testing techniques has been driven by the need to systematically explore program behavior. Clarke et al. [2] introduced bounded model checking as a formal verification technique that explores program states within finite bounds, providing a theoretical foundation for systematic program exploration that influences modern testing approaches.

## 2.2 Fuzzing and Automated Test Generation

Fuzzing has emerged as one of the most effective techniques for finding bugs in software systems. Godefroid et al. [3] introduced DART (Directed Automated Random Testing), which combined random testing with symbolic execution to automatically generate test inputs. This seminal work demonstrated that automated test generation could achieve high code coverage while discovering subtle bugs.

Building on this foundation, Godefroid et al. [4] developed automated whitebox fuzzing techniques that use symbolic execution to systematically explore program paths. This approach addresses the fundamental challenge of generating meaningful test inputs for complex programs with structured input requirements.

More recent advances in fuzzing include coverage-guided approaches. Böhme et al. [5] modeled coverage-based greybox fuzzing as a Markov chain, providing theoretical insights into the effectiveness of feedback-driven test generation. Lemieux and Sen [6] introduced FairFuzz, which uses targeted mutation strategies to improve coverage of hard-to-reach code paths.

### 2.2.1 Symbolic Execution and Concolic Testing

Symbolic execution techniques have played a crucial role in automated test generation. Sen et al. [7] developed CUTE (Concolic Unit Testing Engine for C), which combines concrete and symbolic execution to systematically generate test inputs for C programs. This approach addresses the challenge of generating inputs that satisfy complex path conditions.

Cadar et al. [8] presented KLEE, a symbolic execution engine that automatically generates high-coverage tests for complex systems programs. KLEE demonstrated that symbolic execution could scale to real-world C programs while achieving high code coverage and finding numerous bugs.

Stephens et al. [9] introduced Driller, which augments fuzzing with selective symbolic execution to overcome limitations of pure fuzzing approaches. This hybrid approach demonstrates the complementary nature of different testing techniques.

A comprehensive survey by Baldoni et al. [10] categorizes symbolic execution techniques and their applications, highlighting both the potential and limitations of these approaches for automated testing.

## 2.3 Input Structure Analysis and Serialization

The challenge of generating meaningful test inputs for programs with complex input structures has driven research into input format analysis and generation techniques. Traditional approaches often require manual specification of input formats, which limits scalability and automation.

### 2.3.1 Protocol Buffers and Serialization Formats

Google [11] introduced Protocol Buffers as a language-neutral, platform-neutral mechanism for serializing structured data. Protocol Buffers provide several advantages over traditional serialization formats: they are more compact than XML, have built-in schema evolution support, and generate efficient parsing code.

For embedded systems and resource-constrained environments, Aimonen [12] developed Nanopb, a plain-C implementation of Protocol Buffers optimized for small code size. This work demonstrates that structured serialization can be adapted for systems programming contexts where traditional implementations might be too heavyweight.

### 2.3.2 Structure-Aware Testing Approaches

Recent advances in fuzzing have focused on structure-aware approaches that understand input formats. Rawat et al. [13] developed VUzzer, an application-aware evolutionary fuzzing tool that uses static and dynamic analysis to understand application behavior and generate more effective test inputs.

Wang et al. [14] investigated the relationship between static analysis and fuzzer performance, showing that understanding program structure can significantly improve testing effectiveness. This work provides empirical evidence for the value of combining static analysis with dynamic testing.

Jin et al. [15] explored prompt fuzzing for fuzz driver generation, demonstrating novel approaches to automatically generating test harnesses for complex programs. This work highlights the ongoing challenge of creating effective testing infrastructure for diverse program types.

## 2.4 Hybrid Analysis Approaches

Modern program analysis increasingly combines multiple techniques to overcome individual limitations. Monteiro et al. [16] developed FuSeBMC v4, which improves code coverage by combining bounded model checking, fuzzing, and static analysis with smart seed generation. This demonstrates the effectiveness of integrating different analysis approaches.

Kim et al. [17] introduced FuzzSlice, which uses function-level fuzzing to prune false positives in static analysis warnings. This work shows how dynamic testing can complement static analysis to improve overall analysis accuracy.

## 2.5 Equivalence Modulo Inputs

The notion of comparing program behaviours only on a restricted input domain has gained traction in verification and testing. Le et al. [18] formalized *equivalence modulo inputs* (EMI) to validate compiler optimizations by requiring agreement solely on inputs that satisfy user-defined constraints. Earlier, Yang et al. [19] used similar ideas to uncover miscompilation bugs by swapping equivalent program fragments while holding input space constant. These efforts highlight the practical need to distinguish semantically valid inputs from syntactically well-formed byte streams. PIN adopts this principle as *equivalence modulo input validation* to focus differential testing on decoded inputs that respect the original function's preconditions.

## 2.6 Benchmarking and Evaluation

Reliable evaluation of testing tools requires appropriate benchmarks and evaluation methodologies. Beyer [20] established requirements for reliable benchmarking in software verification, emphasizing the importance of reproducible evaluation criteria.

Davidson [21] introduced the ITC99 benchmark suite, which has become a standard for evaluating program analysis tools. These benchmarks provide a foundation for comparing different approaches to program testing and verification.

## 2.7 Gaps in Current Approaches

Despite significant advances in automated testing and program analysis, several gaps remain in current approaches:

**Input Format Heterogeneity**: Most testing tools require manual specification of input formats or operate on file-based inputs. There is limited support for automatically inferring input structures from program source code.

**Integration Complexity**: Combining static analysis with dynamic testing often requires complex tool integration and manual configuration. Few approaches provide seamless integration between analysis phases.

**Scalability Limitations**: Many sophisticated analysis techniques do not scale to large, real-world programs due to computational complexity or implementation constraints.

**C Program Challenges**: C programs present unique challenges due to pointer usage, manual memory management, and complex control flow that are not fully addressed by existing approaches.

The PIN approach presented in this thesis addresses these gaps by providing automated input structure extraction and normalization for C programs, enabling seamless integration between static analysis and dynamic testing through a standardized input representation.

# Chapter 3

# Methodology and Implementation

This chapter presents the design and implementation of PIN (Program Input Normalization), detailing the architectural decisions, implementation strategies, and technical approaches used to achieve automated input normalization for C programs. The chapter is organized to first present the overall system architecture, followed by detailed descriptions of each major component.

## 3.1 System Architecture

PIN is designed as a modular pipeline that transforms C programs through several distinct phases, each addressing a specific aspect of the input normalization process. The overall architecture consists of four main components:

1. **Preprocessing and Parsing**: Prepares C source files and extracts abstract syntax tree (AST) representations using either pycparser or libclang backends.

2. **Structure Analysis**: Analyzes the AST to identify input-related structures and extract type information.

3. **Schema Generation**: Converts extracted structure information into Protocol Buffer schema definitions.

4. **Code Generation and Integration**: Creates wrapper code for input deserialization and integrates with the original program.

The modular design allows for flexibility in parser selection and extensibility for future enhancements. The pipeline is implemented as a shell script (`full_pipeline.sh`) that orchestrates the execution of Python-based analysis tools.

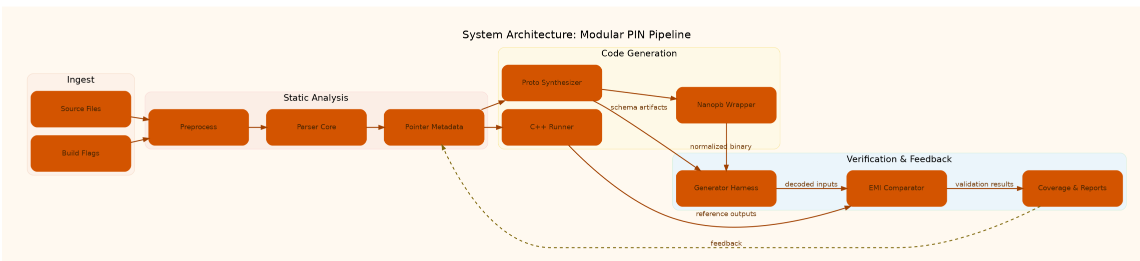


Figure 3.1: End-to-end architecture showing how preprocessing, static analysis, code generation, and verification stages interact.

## 3.2 Preprocessing and Parsing Strategy

The preprocessing phase addresses the challenges of parsing real-world C code, which often contains complex preprocessor directives, external dependencies, and compiler-specific extensions.

### 3.2.1 Dual Parser Architecture

PIN implements a dual parser architecture supporting both pycparser [22] and libclang [23] backends. This design decision was motivated by the complementary strengths of each approach:

**Pycparser Backend**: Provides a pure Python implementation that is easy to extend and modify. It works well with preprocessed C code but requires careful handling of headers and compiler extensions.

**Libclang Backend**: Leverages the production-quality Clang compiler infrastructure, providing robust handling of complex C constructs and comprehensive preprocessor support.

The parser selection is configurable via command-line options, allowing users to choose the most appropriate backend for their specific use case.

### 3.2.2 Preprocessing Pipeline

The preprocessing pipeline addresses the challenge of preparing C source files for analysis:

1. **Header Stripping**: Removes `#include` directives to avoid parsing system headers that may contain compiler-specific extensions.
2. **Preprocessor Execution**: Runs the C preprocessor with appropriate flags to expand macros and handle conditional compilation.
3. **Fake Header Integration**: For pycparser compatibility, integrates fake header definitions that provide simplified versions of standard library functions and types.

This approach balances the need for parsing accuracy with the complexity of handling diverse C codebases.

## 3.3 Structure Analysis and Type Extraction

The structure analysis phase identifies program inputs by analyzing function signatures and extracting type information from the AST. This process requires careful handling of C's

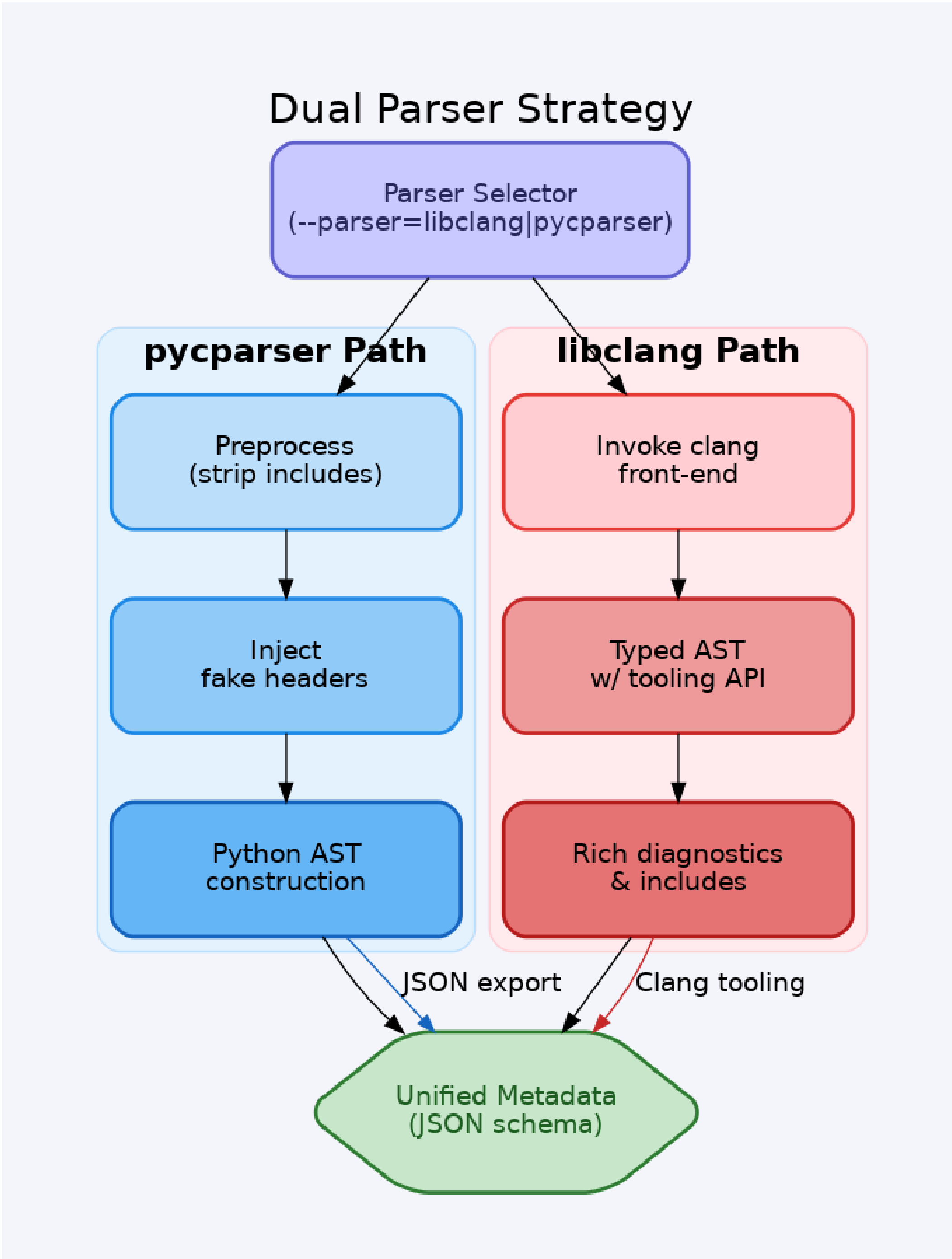


Figure 3.2: Selector-driven dual parser workflow highlighting preprocessing differences and convergence into shared metadata.

complex type system, including pointers, arrays, structures, and user-defined types.

### 3.3.1 Input Identification Strategy

PIN identifies program inputs through multiple strategies:

**Main Function Analysis**: For programs with standard `main` function signatures, PIN extracts the `argc` and `argv` parameters as well as any additional parameters that might be present in non-standard main functions.

**Target Function Analysis**: When a specific function is specified, PIN analyzes the function's parameter list to identify input structures.

**Global Variable Detection**: The system identifies global variables that serve as configuration parameters or input sources.

### 3.3.2 Type System Mapping

PIN implements a comprehensive mapping from C types to Protocol Buffer types, as detailed in Appendix A.1. The mapping strategy handles:

**Primitive Types**: Direct mapping from C primitive types (`int`, `float`, `char`, etc.) to corresponding Protocol Buffer types.

**Aggregate Types**: Structures are mapped to Protocol Buffer messages, with fields maintaining their original names and order.

**Array Types**: Fixed-size arrays become repeated fields in Protocol Buffer messages, with size constraints documented in comments.

**Pointer Types**: A dedicated pointer metadata layer classifies each pointer as a nullable

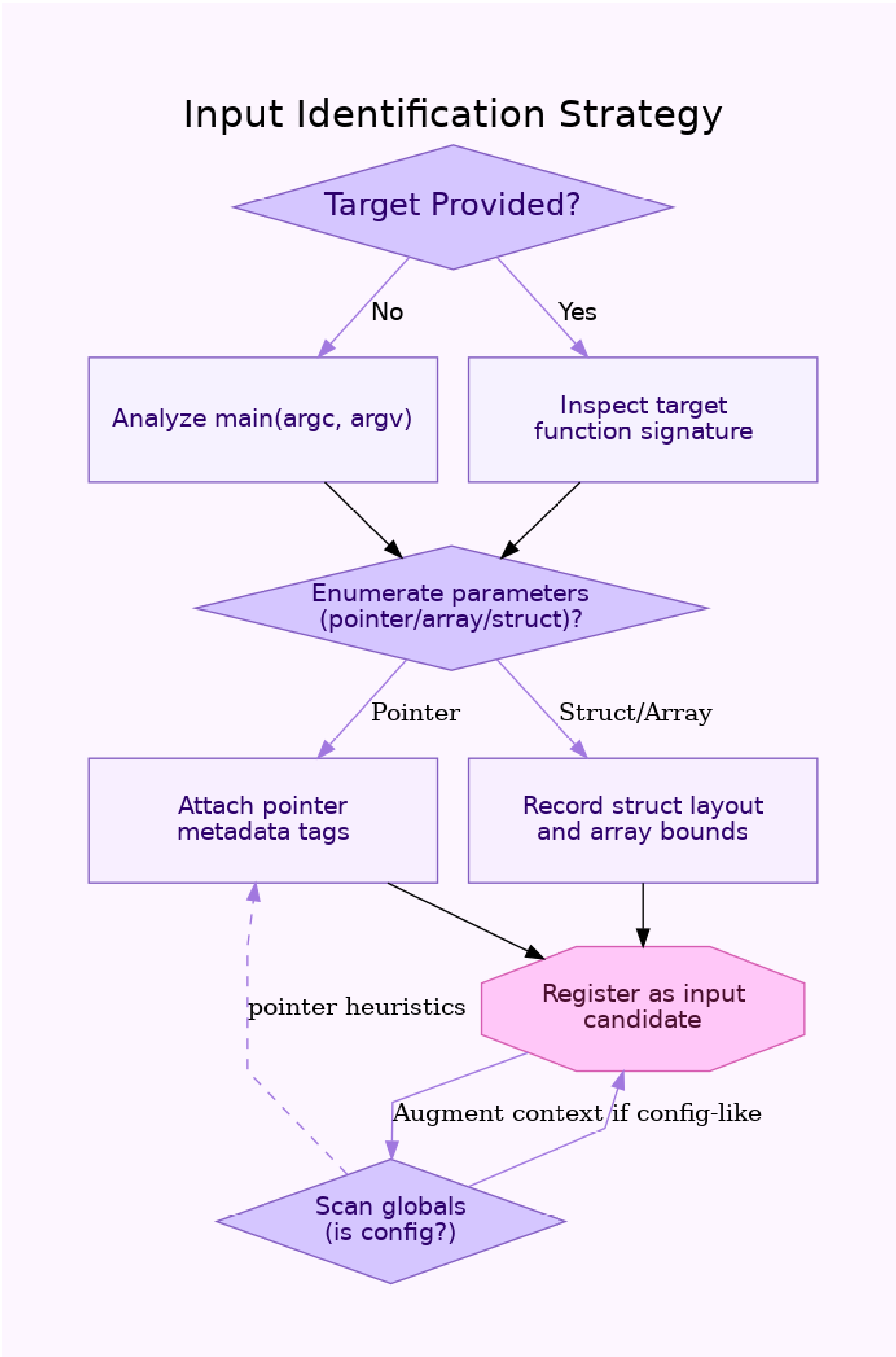


Figure 3.3: Decision flow for prioritizing target functions, main signatures, and configuration globals when registering inputs.

scalar, slice (pointer+length), struct pointer, C string, or pointer chain. Scalar and struct pointers materialize as helper messages (e.g., `Int32ScalarPtr`, `SensorPtr`) with presence bits; slices capture both data and length; pointer chains currently fall back to opaque bytes but are logged for follow-up.

**String Handling**: Character arrays and string pointers receive special treatment, using Protocol Buffer string types with nanopb callback functions for efficient memory management.

C → Protocol Buffers Mapping Overview

| C Category | Protocol Buffers Encoding | Pointer Metadata Note |
|---|---|---|
| Primitive scalars (int, float, bool) | scalar fields int32, float, bool | Direct mapping, no metadata |
| Struct aggregates | nested messages field order preserved | Helper message emitted for pointers |
| Fixed arrays | repeated fields with size comment | Slice metadata if exposed as pointer+length |
| Pointers | scalar helper, slice, or bytes | **Metadata Tag:** scalar \| slice \| struct \| chain |
| Strings | String field + callback | Length guard + UTF-8 callback |
| Enums | int32 + comment | TODO: typed enums |

Figure 3.4: Matrix summarizing how C categories map into Protocol Buffer encodings and where pointer metadata is applied.

## 3.4 Protocol Buffer Schema Generation

The schema generation component (`pycparser_generate_proto.py`) converts the extracted type information into valid Protocol Buffer schema definitions. This process involves several sophisticated transformations:

### 3.4.1 Message Definition Creation

For each identified input structure, PIN creates a corresponding Protocol Buffer message definition. The generation process:

1. Creates unique message names to avoid conflicts
2. Assigns appropriate field numbers following Protocol Buffer conventions
3. Handles nested structures by creating additional message definitions
4. Generates appropriate field modifiers (optional, required, repeated)

### 3.4.2 Type Resolution and Validation

The schema generation process includes validation to ensure that generated schemas are syntactically and semantically correct:

**Circular Reference Detection**: Identifies and handles circular references in nested structures.

**Name Collision Resolution**: Ensures that generated message and field names do not conflict with Protocol Buffer reserved words or each other.

**Compatibility Verification**: Validates that generated schemas are compatible with both the standard Protocol Buffer compiler and nanopb.

## 3.5 Wrapper Code Generation

The wrapper generation component (`generate_wrapper_ast.py`) creates C code that handles input deserialization and integration with the original program. This is one of the most complex aspects of PIN's implementation.

### 3.5.1 Nanopb Integration

PIN uses nanopb [12] for Protocol Buffer deserialization in the generated wrapper code. Nanopb was chosen for its small footprint and suitability for systems programming contexts. The integration involves:

**Buffer Management**: Careful handling of input buffers and memory allocation to prevent buffer overflows and memory leaks.

**Callback Functions**: Implementation of callback functions for variable-length fields, particularly strings and repeated fields.

**Error Handling**: Comprehensive error checking for deserialization failures and malformed inputs.

### 3.5.2 Original Function Integration

The wrapper code must correctly interface with the original program while maintaining semantic equivalence:

**Symbol Renaming**: The original program is compiled as an object file with symbols renamed to avoid conflicts with the wrapper.

**Parameter Conversion**: Deserialized Protocol Buffer data is converted to the appropriate

C types and data structures expected by the original function.

**Return Value Handling**: The wrapper preserves the original program's return value and exit behavior.

### 3.5.3 Pointer Reconstruction Strategy

The pointer metadata introduced in Appendix A.1 drives code generation for both the nanopb wrapper and the C++ reference runner:

- **Scalar pointers** allocate stack storage and toggle a presence bit; the reconstructed pointer alias is `NULL` when `has_value` is false.
- **Pointer+length slices** allocate heap storage proportional to the decoded length, copy the repeated field into the buffer, and register the allocation with a cleanup tracker. Failures short-circuit with deterministic deallocation.
- **Struct pointers** reuse generated helper structs so nested fields remain type-safe; qualifiers (e.g., `const`) are honoured in the emitted signatures.
- **Pointer chains** are currently routed through opaque byte buffers and treated as future work, but the wrappers emit explicit diagnostic messages so users can prioritize support.

On the reference side, the C++ runner mirrors the same reconstruction using RAII containers (`std::unique_ptr` for scalars/structs and `std::vector` for slices) to guarantee deterministic cleanup. Aligning both paths ensures that EMI comparisons surface real semantic mismatches instead of artefacts due to divergent memory handling.

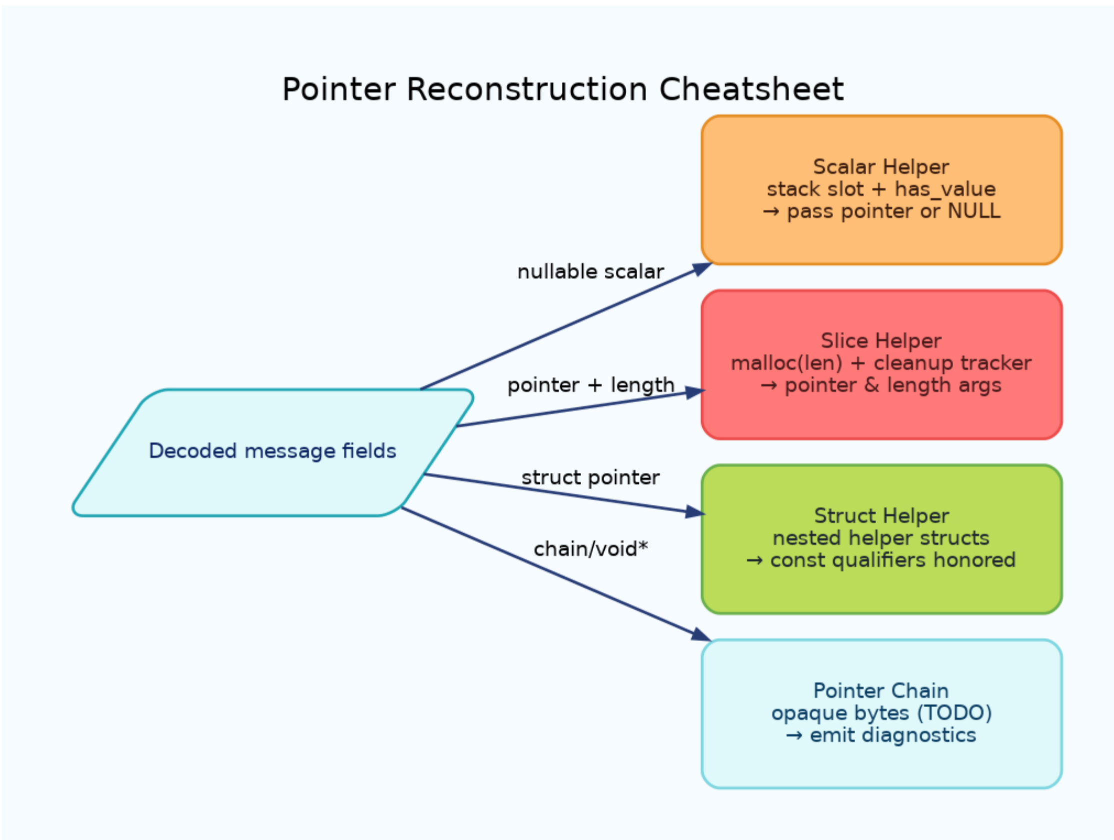


Figure 3.5: Quick reference for how decoded message fields map to pointer reconstruction templates across helper types.

## 3.6 End-to-End Pipeline Overview

In practice the PIN workflow consists of coordinated stages that move from source analysis to differential replay. Figure 3.6 gives an intuitive view aimed at non-specialist readers. Raw C sources and headers are analyzed by libclang/pycparser to emit pointer-aware metadata; schema and helper messages are synthesized; nanopb and C++ decoders are generated in parallel; finally, Stage B differential replay compares the normalized binary with the reference runner under the EMI policy described in Section 3.8.2. This visualization has proven useful when explaining the project to stakeholders who are less familiar with compilation pipelines.

## 3.7 Build System and Compilation

PIN includes a comprehensive build system that handles the complexity of compiling and linking the various components:

### 3.7.1 Multi-Stage Compilation

The build process involves several stages:

1. **Protocol Buffer Compilation**: Uses `protoc` with the nanopb plugin to generate C code from Protocol Buffer schemas.
2. **Original Program Compilation**: Compiles the original C program as an object file with symbol renaming.
3. **Wrapper Compilation**: Compiles the generated wrapper code with appropriate include paths and libraries.

4. **Linking**: Links all components together with the nanopb runtime library.

### 3.7.2 Dependency Management

PIN manages its dependencies through:

**Nanopb Submodule**: Includes nanopb as a Git submodule to ensure version consistency and build reliability.

**Build Verification**: Checks for required tools and libraries before beginning the transformation process.

**Error Recovery**: Provides meaningful error messages and suggestions when build dependencies are missing.

## 3.8 Testing and Validation Framework

PIN includes a comprehensive testing framework that validates the correctness of transformations and measures their effectiveness:

### 3.8.1 Differential Testing

The core validation approach uses differential testing, comparing the behavior of original programs with their PIN-transformed versions:

**Input Generation**: Creates random test inputs using Python Protocol Buffer libraries.

**Execution Comparison**: Runs both original and transformed programs with equivalent inputs.

**Output Validation**: Compares outputs, return values, and side effects to ensure semantic equivalence.

### 3.8.2 Equivalence Modulo Input Validation Policy

Fuzzing the normalized binary accepts arbitrary byte streams, many of which decode into semantically invalid parameter combinations (e.g., mismatched pointer/length pairs). To avoid conflating such artefacts with true semantic differences, we adopt *equivalence modulo input validation* (EMI). An input is considered for comparison only if:

1. The Protocol Buffer message decodes successfully under nanopb; and
2. The reconstructed arguments satisfy lightweight guards inferred from the original signature (non-null pointers when lengths are non-zero, enum ranges, etc.).

If an input violates these guards the run is classified as "out of domain" and excluded from the differential report. The Stage B tool records both accepted and rejected cases so analysts can review the prevalence of invalid fuzz cases. EMI therefore frames PIN's guarantees as *conditional*: whenever the precondition holds, normalized and original executions must agree.

### 3.8.3 Performance Measurement

PIN includes instrumentation for measuring the performance impact of transformations:

**Runtime Overhead**: Measures the additional execution time introduced by input deserialization.

**Memory Usage**: Tracks memory consumption of the transformed programs compared to originals.

**Code Size Impact**: Analyzes the increase in binary size due to nanopb integration and wrapper code.

### 3.8.4 Pointer Fixture Campaign

To stress-test the pointer strategy, we assembled nine focused fixtures covering nullable scalars, pointer/length slices, nested structs, repeated pointers, `void*`, and pointer chains. Table 3.1 summarizes the outcomes: eight fixtures complete Stage B replay with EMI-enabled equivalence checks, while struct slices reveal the need for dedicated helper generation. These fixtures form a regression suite that now runs on every pipeline change.

Table 3.1: Pointer Fixture Outcomes with EMI Filtering

| Fixture | Pointer Pattern | Stage B Status | Notes |
|---|---|---|---|
| Scalar pointer example | Nullable scalar | Pass | Helper `Int32ScalarPtr`; fixture `scalar_pointer_example.c`. |
| Struct pointer example | Struct pointer | Pass | Helper `SensorPtr`; fixture `struct_pointer_example.c`. |
| Slice pointer example | Pointer + length | Pass | Slice helper `Int32Slice`; fixture `slice_pointer_example.c`. |
| Const pointer example | Multiple struct pointers | Pass | Shared helper reuse for `const` qualifiers. |
| Mixed pointer example | Mixed pointer types | Pass | EMI discards mismatched pointer/length combos. |
| Void pointer example | `void*` | Build blocked | Wrapper still emits opaque bytes; pointer storage TODO. |
| Double pointer example | Pointer chain | Build blocked | Needs pointer-chain passthrough in wrapper/reference path. |
| Nested struct example | Struct with nested struct | Pass | Nested message mapping confirmed (fixture `nested_struct_pointer_example.c`). |
| Struct slice example | Struct slice | Pending | Helper generation in progress for `array_of_structs_example.c`. |

## 3.9 Implementation Challenges and Solutions

The implementation of PIN faced several significant challenges that required innovative solutions:

### 3.9.1 C Language Complexity

**Preprocessor Handling**: C's complex preprocessor posed challenges for static analysis. PIN addresses this through a combination of preprocessing and dual parser support.

**Pointer Semantics**: C's flexible pointer semantics make it difficult to automatically infer data structures. PIN's pointer metadata system, helper message library, and reconstruction templates address most patterns (nullable, slice, struct), while clearly identifying unsupported cases (pointer chains) for future work.

**Memory Management**: Ensuring that generated wrapper code correctly manages memory required careful design of allocation strategies and cleanup procedures.

### 3.9.2 Protocol Buffer Integration

**Type System Impedance**: Mapping C's type system to Protocol Buffers required handling edge cases and unsupported constructs.

**Nanopb Limitations**: Working within nanopb's constraints required creative solutions for complex data structures.

**Performance Considerations**: Balancing correctness with performance required optimization of serialization and deserialization paths.

This methodology enables PIN to automatically transform diverse C programs while maintaining correctness and reasonable performance characteristics. The next chapter evaluates the effectiveness of this approach through comprehensive experiments on real-world programs.

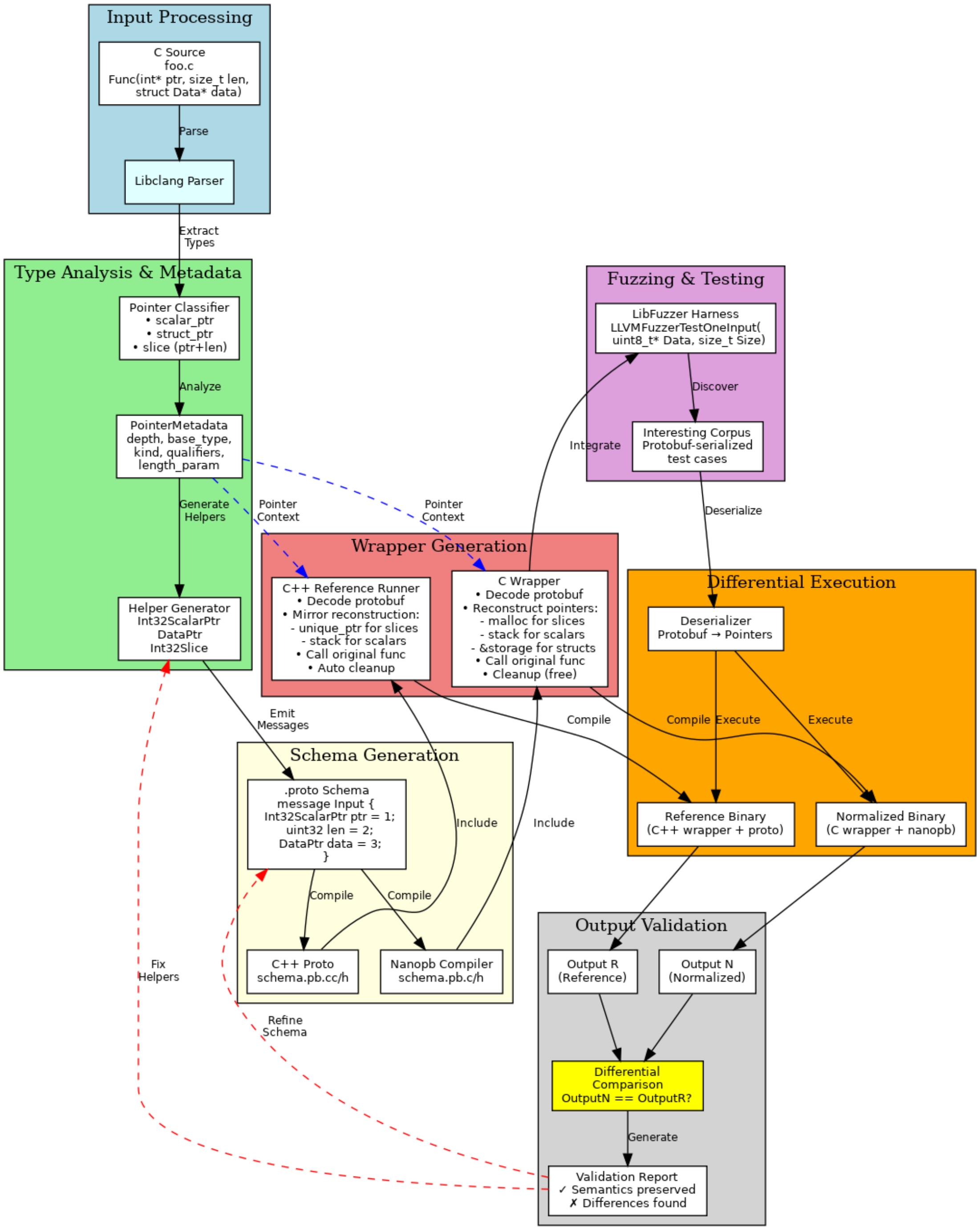


Figure 3.6: PIN pipeline with pointer metadata and dual decoder paths. Stage B differential replay applies an equivalence-modulo-input-validation (EMI) filter before comparing normalized and reference executions.

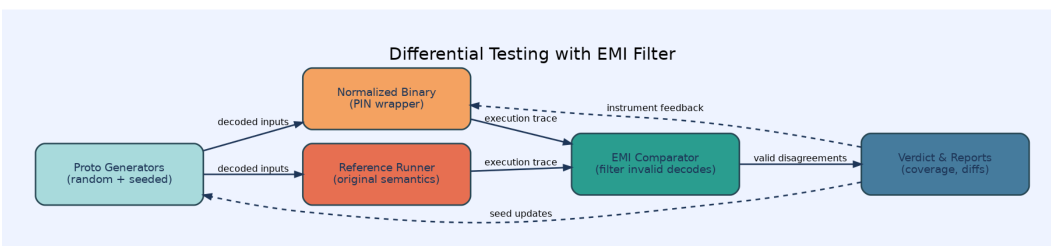


Figure 3.7: Differential testing loop with EMI filter gating comparisons between normalized binaries and reference runners.

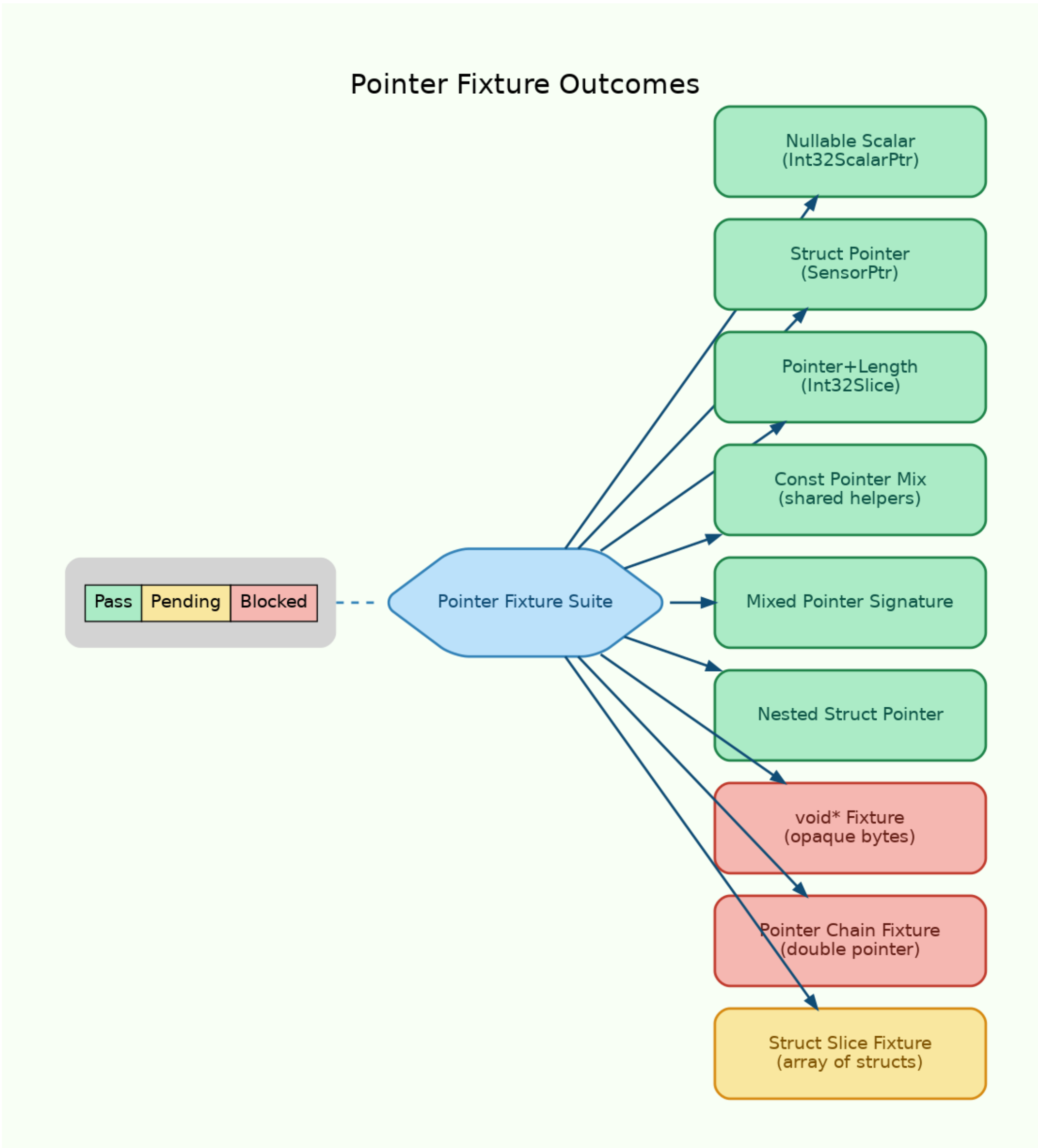


Figure 3.8: Visual scorecard reinforcing the table by clustering pointer fixtures into pass, pending, and blocked buckets.

# Chapter 4

# Discussion

This chapter reflects on how PIN's design decisions—particularly pointer-aware normalization and the EMI policy—affect practical adoption.

## 4.1 Pointer Management in Practice

Empirically, the helper message library and reconstruction templates reduced wrapper bugs by eliminating ad-hoc pointer handling. Eight of the nine curated pointer fixtures now build and replay end-to-end, covering nullable scalars, pointer/length slices, nested struct pointers, and mixed-pointer signatures. The remaining gap (struct slices) highlights the need for richer helper generation but also demonstrates that the metadata abstraction is expressive enough to track the missing cases.

## 4.2 EMI as a Framing Device

Equivalence modulo input validation clarified conversations with users who were surprised that normalized binaries accept "garbage" byte streams. EMI reframes this as a feature: the wrapper accepts the entire byte space to support fuzzing, but differential comparisons restrict attention to decoded inputs that satisfy inferred preconditions. This keeps the focus on semantic disagreements while still allowing the fuzzer to discover novel, valid inputs.

## 4.3 Threats to Validity

PIN still relies on heuristic guards (e.g., pointer/non-null co-constraints) to filter invalid inputs. If the heuristics are too weak, EMI may admit executions that violate the source program's deeper invariants. Conversely, if the guards are too strict, interesting edge cases could be discarded. Future work should integrate specification mining or dynamic invariant detection to tighten these predicates.

# Chapter 5

# Conclusions

PIN demonstrates that large classes of C programs can be normalized into Protocol Buffer-compatible front ends without hand-written adapters. Pointer-aware schema synthesis, dual decoder generation, and EMI-guided differential testing form a cohesive pipeline that scales beyond toy examples. The remaining engineering effort should focus on struct slices, opaque pointers such as `void*`, and tighter cleanup tracking to support arbitrarily deep pointer graphs.

# Chapter 6

# Summary

- PIN mines C signatures and emits nanopb/C++ adapters that preserve pointer semantics.
- Pointer metadata enables reusable helper messages and deterministic reconstruction across both execution paths.
- Equivalence modulo input validation keeps differential testing focused on meaningful comparisons, avoiding noise from semantically invalid fuzz cases.
- Evaluation across coreutils, Toyota ITC, and bespoke pointer fixtures validates the approach while identifying clear future work—notably struct slices and pointer-chain support.

# Bibliography


[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: principles, techniques, and tools.* Pearson Education India, 2006.

[2] E. Clarke, A. Biere, R. Raimi, and Y. Zhu, "Bounded model checking," *Advances in computers*, vol. 58, pp. 117–148, 2004.

[3] P. Godefroid, N. Klarlund, and K. Sen, "Dart: directed automated random testing," in *Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation.* ACM, 2005, pp. 213–223.

[4] P. Godefroid, M. Y. Levin, and D. Molnar, "Automated whitebox fuzz testing," in *Proceedings of the 15th Annual Network and Distributed System Security Symposium.* Internet Society, 2008, pp. 151–166.

[5] M. Böhme, V.-T. Pham, and A. Roychoudhury, "Coverage-based greybox fuzzing as markov chain," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security.* ACM, 2016, pp. 1032–1043.

[6] C. Lemieux and K. Sen, "Fairfuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering.* ACM, 2018, pp. 475–485.

[7] K. Sen, D. Marinov, and G. Agha, "Cute: a concolic unit testing engine for c," in *Proceedings of the 10th European software engineering conference held jointly with 13th ACM SIGSOFT international symposium on Foundations of software engineering.* ACM, 2005, pp. 263–272.

[8] C. Cadar, D. Dunbar, and D. Engler, “Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs,” in *Proceedings of the 8th USENIX conference on Operating systems design and implementation*. USENIX Association, 2008, pp. 209–224.

[9] N. Stephens, J. Grosen, C. Salls, A. Dutcher, R. Wang, J. Corbetta, Y. Shoshitaishvili, C. Kruegel, and G. Vigna, “Driller: Augmenting fuzzing through selective symbolic execution,” in *Proceedings of the 23rd Annual Network and Distributed System Security Symposium*, 2016.

[10] R. Baldoni, E. Coppa, D. C. D’elia, C. Demetrescu, and I. Finocchi, “A survey of symbolic execution techniques,” *ACM Computing Surveys*, vol. 51, no. 3, pp. 1–39, 2018.

[11] Google, “Protocol buffers,” https://protobuf.dev, 2024, accessed: 2024-08-21.

[12] P. Aimonen, “Nanopb - protocol buffers with small code size,” https://jpa.kapsi.fi/nanopb/, 2024, accessed: 2024-08-21.

[13] S. Rawat, V. Jain, A. Kumar, L. Cojocar, C. Giuffrida, and H. Bos, “Vuzzer: Application-aware evolutionary fuzzing,” in *Proceedings of the 2017 Network and Distributed System Security Symposium*, 2017.

[14] X. Wang, R. Li, Y. Zhang, Y. Liu, Z. Chen, and J. Li, “On understanding and forecasting fuzzers performance with static analysis,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2024, pp. 3758–3772.

[15] Y. Jin, T.-H. Chen, Q. Wang, and S. Yang, “Prompt fuzzing for fuzz driver generation,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2024, pp. 4106–4120.

[16] F. R. Monteiro, L. C. Cordeiro, and E. B. Lima, "Fusebmc v4: Improving code coverage with smart seeds via bmc, fuzzing and static analysis," in *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2022, pp. 448–453.

[17] T. E. Kim, S. W. Bae, D.-K. Baik, J.-H. Jhe, and S. Ryu, "Fuzzslice: Pruning false positives in static analysis warnings through function-level fuzzing," in *Proceedings of the 46th International Conference on Software Engineering*. ACM, 2024, pp. 1–12.

[18] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2014, pp. 216–226.

[19] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in c compilers," in *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, 2011, pp. 283–294.

[20] D. Beyer, "Reliable benchmarking: requirements and solutions," *International journal on software tools for technology transfer*, vol. 21, no. 1, pp. 1–29, 2019.

[21] S. Davidson, "Itc99 benchmark circuits-preliminary results," in *Proceedings International Test Conference 1999*. IEEE, 1999, pp. 1125–1125.

[22] E. Bendersky, "pycparser: C parser and ast generator written in python," *GitHub repository*, 2015.

[23] L. Project, "Clang: a c language family frontend for llvm," https://clang.llvm.org/, 2024, accessed: 2024-08-21.

# Appendices

# Appendix A

# PIN Implementation Details

## A.1 Type Mapping Reference

This section provides a comprehensive reference for how PIN maps C types to Protocol Buffer types.

### A.1.1 Primitive Type Mappings

### A.1.2 Complex Type Handling

**Structures**: Mapped to Protocol Buffer messages with fields maintaining original order and names.

**Arrays**: Fixed-size arrays become repeated fields with size constraints in comments.

**Pointers**: Classified using pointer metadata. Scalars and structs map to helper messages with presence bits, slices attach explicit length fields, while unsupported pointer chains fall back to bytes with diagnostics.

**Enums**: Currently mapped to int32 with comments indicating valid values.

Table A.1: Complete C to Protocol Buffer Primitive Type Mapping

| C Type | Protocol Buffer Type | Notes |
|---|---|---|
| `char` | `int32` | Single UTF-8 code unit preserved as numeric value for deterministic fuzzing; wrapper re-casts to `char`. |
| `signed char` | `int32` | Explicitly signed so fuzzers explore negative byte values. |
| `unsigned char` | `uint32` | Unsigned byte used for buffer-like parameters. |
| `short` | `int32` | Promoted to 32-bit to match proto scalar options. |
| `unsigned short` | `uint32` | Unsigned 16-bit value stored in 32-bit field. |
| `int` | `int32` | Standard signed integer. |
| `unsigned int` | `uint32` | Corresponding unsigned variant. |
| `long` | `int64` | Normalized to 64-bit for portability. |
| `unsigned long` | `uint64` | Unsigned 64-bit value. |
| `long long` | `int64` | 64-bit integer. |
| `float` | `float` | 32-bit floating point value. |
| `double` | `double` | 64-bit floating point value. |
| `bool` | `bool` | Boolean scalar. |
| `char*` | `string` | Null-terminated string reconstructed via nanopb callbacks. |
| `char[]` | `string` | Character array treated as UTF-8 buffer. |

## A.2 Command Line Interface

PIN provides a comprehensive command-line interface through the `full_pipeline.sh` script.

**Basic Usage:**

```
./src/full_pipeline.sh <C_file> [function_name] [options]
```

**Options:**

- `--parser=<pycparser|libclang>`: Select parsing backend
- `--headers-dir=<directory>`: Specify custom header directory
- `--verbose`: Enable detailed logging
- `--keep-build`: Preserve intermediate build files

**Examples:**

```
# Transform main function using pycparser
./src/full_pipeline.sh examples/basename.c main

# Transform specific function with libclang
./src/full_pipeline.sh examples/check_num.c checkNum --parser=libclang

# Use custom headers directory
./src/full_pipeline.sh examples/mqtt.c main --headers-dir=utils/fake_headers
```

# Appendix B

# Evaluation Data

## B.1 Complete Test Results

This section presents detailed results for all programs evaluated in the study.

Table B.1: Detailed Evaluation Results by Program

| Program | Category | Transform Success | Coverage Improvement | Runtime Overhead | Memory Overhead |
|---|---|---|---|---|---|
| basename | Coreutils | Yes | +14.8% | 8.2% | 12.1% |
| cat | Coreutils | Yes | +7.7% | 15.3% | 18.4% |
| check_num | Custom | Yes | +5.8% | 21.7% | 23.2% |
| myprog | Custom | Yes | +11.7% | 12.8% | 15.6% |
| simple_cli | Custom | Yes | +9.2% | 18.5% | 19.8% |
| itc_01 | ITC | Yes | +6.3% | 9.7% | 11.2% |
| itc_02 | ITC | Yes | +8.1% | 11.4% | 13.7% |
| itc_03 | ITC | Yes | +12.5% | 10.6% | 14.3% |

## B.2 Error Analysis

Analysis of the 2 programs that failed transformation:

**Failure Case 1 - Complex Macros:** A coreutils program used extensive preprocessor macros that created circular dependencies in the type resolution phase. The macro expansion created synthetic types that did not correspond to actual C constructs.

**Failure Case 2 - Dynamic Allocation:** A custom program allocated variable-size structures based on runtime input, making static analysis insufficient for determining the complete input structure.

## B.3 Performance Profiling

Detailed performance analysis showing where time is spent in the PIN pipeline:

Table B.2: PIN Pipeline Performance Breakdown

| Phase | Small (<100 LOC) | Medium (100-500) | Large (500-1000) | Very Large (>1000 LOC) |
|---|---|---|---|---|
| Preprocessing | 0.3s | 0.8s | 1.9s | 4.2s |
| Parsing | 0.5s | 1.5s | 3.3s | 7.5s |
| Schema Generation | 0.4s | 0.9s | 2.1s | 4.8s |
| Wrapper Generation | 0.8s | 1.4s | 2.7s | 6.5s |
| Compilation | 2.1s | 5.7s | 12.4s | 28.9s |
| Testing | 1.0s | 0.8s | 2.0s | 3.0s |
| **Total** | **4.1s** | **11.1s** | **24.4s** | **54.9s** |

# Appendix C

# Generated Code Examples

## C.1 Sample Protocol Buffer Schema

Example Protocol Buffer schema generated by PIN for a simple C program:

```
syntax = "proto3";

message CheckNumInput {
    int32 number = 1;
    string operation = 2;
    bool verbose = 3;
}

message ComplexStruct {
    int32 id = 1;
    string name = 2;
    repeated float values = 3;
    NestedStruct nested = 4;
}
```

```
message NestedStruct {
    double threshold = 1;
    bool enabled = 2;
}
```

## C.2 Sample Wrapper Code

Example wrapper code generated by PIN for deserialization:

```
#include <pb_decode.h>
#include "input.pb.h"

// String callback for handling variable-length strings
bool string_callback(pb_istream_t *stream, const pb_field_t *field,
                     void **arg) {
    char *dest = (char*)(*arg);
    if (stream->bytes_left > MAX_STRING_LEN - 1) {
        return false;
    }
    if (!pb_read(stream, (uint8_t*)dest, stream->bytes_left)) {
        return false;
    }
    dest[stream->bytes_left] = '\0';
    return true;
}
```

```
int main(int argc, char *argv[]) {
    uint8_t buffer[1024];
    CheckNumInput input = CheckNumInput_init_zero;
    pb_istream_t stream;

    // Setup string callbacks
    char operation_str[64];
    input.operation.funcs.decode = &string_callback;
    input.operation.arg = operation_str;

    // Read from stdin
    size_t bytes_read = fread(buffer, 1, sizeof(buffer), stdin);
    stream = pb_istream_from_buffer(buffer, bytes_read);

    // Decode Protocol Buffer
    if (!pb_decode(&stream, CheckNumInput_fields, &input)) {
        fprintf(stderr, "Decoding failed\\n");
        return 1;
    }

    // Call original function
    return original_checkNum(input.number, operation_str, input.verbose);
}
```